Multilayer Perceptron

Monday, 19 October 2026 4:07 PM

Deep learning: 1/ GPUs enabled performing calculations in parallel → Lightning fast (Jensen)
2/ Mathematical innovations in DNNs
3/ Easy access to data
4/ Machine learning libraries

When is Deep learning useful:

- Tasks that cannot have a generative solving algorithms ("difficult tasks")
- Multiple processing stages that are jointly optimized ("End-to-end learning")
- Lots of data
- High-dimensional data
- Differentiable loss functions

Supervised Learning:

- input → Learn model → Map output

We have already pairs of (input, output)

- Classification { Binary ($y=0$, $y=1$); Multiclass }
- Regression

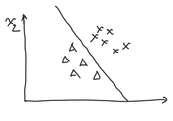
Perceptron

$$a = \sum_i x_i \cdot w_i + b \quad \Rightarrow \quad y = \begin{cases} 0, & a<0 \\ 1, & a \geq 0 \end{cases}$$

$$= x^T w + b \quad \left| \; x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad w = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix} \right.$$

$$a = x_1 w_1 + x_2 w_2 + b = 0$$

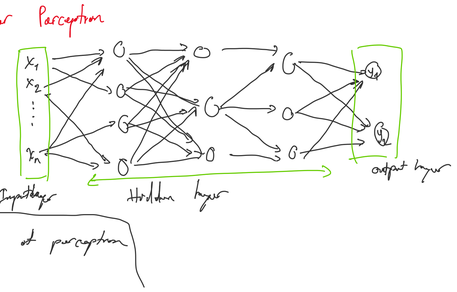
$$\hookrightarrow x_2 = \frac{-b - x_1 w_1}{w_2}$$



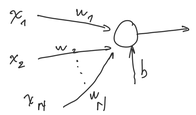
⇓

Linear: main drawback of perceptron

Multilayer Perceptron



Artificial Neurons

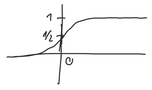


We optimized by changing the weights w_i and bias b

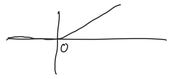
pre-activation: $z = \sum_i x_i w_i + b$

$a = f(z)$ ← activation; → activation function

e.g. • sigmoid $a = f(z) = \frac{1}{e^{-z}+1}$



• Rectified Linear Unit (ReLU): $a = f(z) = \max(0, z)$



$$\bar{z}^l = W^l \cdot \bar{a}^{l-1}$$
$$\Rightarrow \bar{a}^l = f(\bar{z})$$

Computing via vectors, $l-1$ means vectors

Training DNNs

- Applies to all forms of DNNs, not just multilayer perceptron

$(\bar{x}_n, \bar{y}_n)$

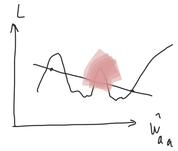
Training the network with loss function $L(\hat{y}_i, y_i)$ → other

↓ optimizes

- $\hat{y}_n = f_W(\bar{x}_n)$

e.g. MSE: $\frac{1}{n} \sum (\hat{y}_i - y_i)^2$

highly { non-linear; non-convex }



If we draw a line between 2 points ⇒ a convex function would all the points below it

Gradient Descent



• : Local optima

Moving against the gradient

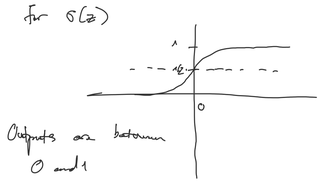
$w^i = w^{i-1} - \alpha \frac{dL}{dW}$

Not guaranteed to find the global minimum

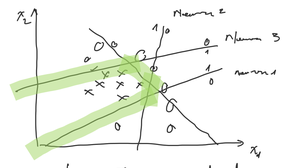
↳ Can reduce # local minima by choosing
- Non-linearities
- Learning Architectures
- etc.

How non-linearities help

For $\sigma(z)$



Outputs are between 0 and 1



↳ • The sigmoid draws this nonlinear boundary
• Also works with multiple clusters

$$\hat{y} = f_L(W_L(f_{L-1} \cdots f_1(W_1 \bar{x}) \cdots))$$

$$\bar{x} \xrightarrow{W^1} \bar{z}^1 \xrightarrow{f^1} \bar{a}^1 \xrightarrow{W_2} \bar{z}^2 \xrightarrow{f^2} \bar{a}^2 \longrightarrow \cdots$$

$$\frac{\partial}{\partial x} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Going to the next layer

$$\frac{\partial L}{\partial a_i^l} = \sum_j \frac{\partial L}{\partial z_j^{l+1}} \cdot \frac{\partial z_j^{l+1}}{\partial a_i^l} \cdot \frac{\partial z_j^{l+1}}{\partial w_{ij}^{l+1}}$$

$$\frac{\partial L}{\partial w_{ij}^l} = \frac{\partial L}{\partial z_i^l} \cdot \frac{\partial z_i^l}{\partial w_{ij}^l}$$

a_j^{l-1}

$$\frac{\partial L}{\partial z_i^l} = \frac{\partial L}{\partial a_i^l} \cdot \frac{\partial a_i^l}{\partial z_i^l}$$

$f'(z_i^l)$

Back-propagation many layers ⇒ Many scalars ⇒ { Vanishing Gradient; Exploding Gradient } ⇒ Need solution